@AnkushKumar

40 SQL PRACTICE QUESTIONS

Basic to Advanced — Learn by Solving Real Problems



Q1. From Customers, display customers whose Score is greater than 500. Show CustomerID, FirstName, Country and Score.

```
select CustomerID, FirstName, Country, Score
from customers
where Score > 500;
```

Q2. Find employees whose Salary is greater than 50,000 and whose Department is Marketing. Display their names, department and salary.

```
select FirstName, department, salary
from employees
where salary > 50000 and department = "Marketing";
```

Q3. Find orders whose OrderStatus is Shipped or Delivered. Display OrderID, OrderStatus and Sales.

```
select OrderID, OrderStatus, Sales
from Orders
where OrderStatus IN ('Shipped', 'Delivered');
```

Q4. Display products whose Category is either Accessories or Clothing.

```
select Product
from salesdb.Products
where Category IN ('Accessories', 'Clothing');
```

Q5. Find employees whose salary is between 55,000 and 80,000, inclusive.

```
select *
from Employees
where Salary between 55000 and 80000;
```

Q6. Find customers whose FirstName starts with the letter M.

```
select FirstName from customers
where FirstName like "M%";
```

Q7. Find records where LastName is missing. Display the customer ID and first name.

```
select customers.customerid, FirstName from
customers
where lastname is null;
```

Q8. Count the number of orders for each OrderStatus. Display the status and order count.

```
select OrderStatus, count(Orderid) Total_Orders
from orders
group by OrderStatus;
```

Q9. calculate total sales and average sales for each ProductID.

```
select ProductID, sum(sales) Total_Sales, round(avg(Sales),0) Avg_Sales
from orders
group by ProductID;
```

Q10. Find CustomerID values whose total Sales is greater than 30. Display the customer ID and total sales.

```
select CustomerID, sum(Sales) Total_Sales
from orders
group by CustomerID
having Total_Sales > 30;
```

Q11. Display OrderID, CustomerID, customer FirstName and Sales for orders with matching customer records.

```
select o.OrderID, o.CustomerID, c.FirstName, o.Sales
from salesdb.orders as o
Inner join salesdb.customers as c
on o.CustomerID = c.CustomerID;
```

Q12. Display OrderID, product name, category, quantity and sales.

```
select o.OrderID, p.Product, p.Category, o.Quantity, o.Sales
from salesdb.orders as o
left join salesdb.products as p
on o.productid = p.productid;
```

Q13. Use Orders.SalesPersonID and Employees.EmployeeID to display each order's ID, sales amount and salesperson's name.

```
select o.OrderID, o.Sales, e.firstname
from salesdb.orders as o
left join salesdb.employees as e
on o.salespersonid = e.employeeid;
```

Q14. Display customer names and their order IDs, showing only customers who have matching orders.

```
select c.FirstName, o.OrderID
from Orders as o
join customers as c
on o.CustomerID = c.CustomerID;
```

Q15. Display each product name and its associated order IDs. Include the product category.

```
select p.Product, o.OrderId, p.category
from products as p
left join orders o
on p.productid = o.productid;
```

Q16. Display all customers and their order IDs. Customers without orders must also appear.

```
select c.FirstName, o.OrderID
from customers as c
left join orders as o
on c.customerid = o.customerid;
```

Q17. Find customers who have no matching records in Orders.

```
select c.FirstName, o.OrderID
from customers as c
left join orders as o
on c.customerid = o.customerid
where o.OrderID is null;
```

Q18. Display all products that have no matching order records. Return the product ID, product name and category.

```
select p.Productid, p.Product, p.Category
from salesdb.products as p
left join orders as o
on p.productid = o.productid
where o.productid is null;
```

Q19. Write a query using RIGHT JOIN to show all products and their matching order IDs, including products without orders.

```
select p.Product, o.OrderID
from orders as o
right join salesdb.products as p
on p.productid = o.productid;
```

Q20. Display orders with OrderStatus = 'Delivered', including the customer name and product name.

```
select o.OrderID, c.FirstName, p.product, o.OrderStatus from orders as o
left join customers as c
on o.customerid = c.customerid
left join salesdb.products as p
on o.productid = p.productid
where OrderStatus = 'Delivred';
```

Q21. Join Orders, Customers and Products. Display OrderID, customer first name, product name, quantity and sales.

```
select o.OrderID, c.FirstName, p.Product, o.Quantity, o.Sales
from orders as o
left join customers as c
on o.customerid = c.customerid
left join salesdb.products as p
on o.productid = p.productid;
```

Q22. Display orders for customers from the USA. Show customer name, country, order ID and sales.

```
select c.FirstName, c.Country, o.OrderID, o.Sales
from orders as o
left join salesdb.customers as c
on o.customerid = c.customerid
where Country = 'USA';
```

Q23. Calculate total sales for each customer. Display CustomerID, customer first name and total sales.

```
select c.CustomerID, c.FirstName, sum(o.Sales) Total_Sales
from orders as o
left join salesdb.customers as c
on o.customerid = c.customerid
group by c.CustomerID, c.FirstName;
```

Q24. Find customers whose total sales across their orders exceed 30. Display customer name and total sales.

```
select c.FirstName, Sum(o.Sales) Total_Sales
from orders as o
left join salesdb.customers as c
on o.customerid = c.customerid
group by c.Customerid, c.FirstName
having sum(o.Sales) > 30;
```

Q25. Count the number of orders handled by each employee. Display employee ID, employee name and order count.

```
select e.Employeeid, e.FirstName, count(o.OrderID) Total_Orders
from salesdb.employees as e
left join salesdb.orders as o
on e.employeeid = o.salespersonid
group by Employeeid, e.FirstName;
```

Q26. Use Employees.ManagerID and Employees.EmployeeID to display each employee's name alongside their manager's name. Keep employees without a manager in the result.

```
select e.EmployeeID, e.FirstName AS Employee_Name,
m.FirstName AS Manager_Name
FROM salesdb.Employees AS e
left join salesdb.Employees AS m
on e.ManagerID = m.EmployeeID;
```

Q27. For each product category, calculate total quantity sold and total sales. Join Orders with Products.

```
select p.Category, sum(o.Quantity) T_Quantity, sum(o.Sales) T_Sales
from orders as o
left join salesdb.products as p
on o.productid = p.productid
group by Category;
```

Q28. Count orders for every customer, including customers with zero orders. Display the customer name and order count.

```
select c.CustomerID, c.Firstname, count(o.OrderID) Total_Orders
from customers as c
left join orders as o
on c.customerid = o.customerid
group by c.Firstname, c.CustomerID;
```

Q29. Compare Orders and OrdersArchive using OrderID. Show all IDs, including records that exist in only one table. Use FULL OUTER JOIN if your database supports it.

```
select o.OrderID AS Current_OrderID, oa.OrderID AS Archive_OrderID
from salesdb.Orders as o
left join salesdb.orders_archive as oa
on o.OrderID = oa.OrderID

union

select o.OrderID AS Current_OrderID, oa.OrderID AS Archive_OrderID
from salesdb.Orders as o
right join salesdb.orders_archive as oa
on o.OrderID = oa.OrderID
where o.OrderID is null;
```

Q30. Join Orders and OrdersArchive on OrderID. Display IDs where the current OrderDate differs from the archived OrderDate. Return both dates so you can compare them.

```
select o.OrderID, o.OrderDate as Current_OrderDate,
oa.OrderDate as Archive_OrderDate
from salesdb.orders as o
inner join salesdb.orders_archive as oa
on o.orderid = oa.orderid
where o.OrderDate <> oa.OrderDate;
```

Q31. Join Customers and Orders. Display each customer's name, country, total sales and total number of orders. Include only customers whose total sales exceed 20. Sort from highest to lowest total sales.

```
select c.CustomerID c.FirstName, c.Country, sum(o.Sales) Total_Sales,
count(o.OrderID) Total_Orders
from salesdb.customers as c
inner join salesdb.orders as o
on c.customerid = o.customerid
group by c.FirstName, c.Country, c.CustomerID
having sum(o.Sales) > 20
order by Total_Sales desc;
```

Q32. Join Products and Orders. For each product, calculate total quantity sold, total sales and average sales per order. Show only products whose total quantity sold is greater than 1.

```
select p.ProductID, p.Product, sum(o.quantity) T_Quantity, sum(o.sales)
T_Sales, round(avg(o.sales),0) Avg_Sales
from salesdb.products as p
left join salesdb.orders as o
on p.productid = o.productid
group by p.ProductID, p.Product
having sum(o.quantity) >1;
```

Q33. Join Employees and Orders. Calculate total sales for each salesperson, considering only orders with status Delivered. Show salespeople whose delivered sales exceed 20.

```
select e.EmployeeID, e.FirstName, sum(o.sales) Total_Sales
from salesdb.employees as e
inner join salesdb.orders as o
on o.salespersonid = e.employeeid
where o.orderstatus = 'Delivered'
group by e.FirstName, e.EmployeeID
having sum(o.sales) >20;
```

Q34. Join Customers and Orders. Display each customer's name, score and total sales. Filter for customers whose score is at least 500 and whose total sales exceed 20.

```
select c.CustomerID, c.FirstName, c.Score, sum(o.Sales) Total_Sales
from salesdb.orders as o
inner join salesdb.customers as c
on o.customerid = c.customerid
where c.Score >=500
group by c.CustomerID, c.FirstName, c.Score
having sum(o.sales) >20;
```

Q35. Join Orders and OrdersArchive on OrderID. Find records where the Sales value differs between the two tables. Display the order ID, current sales and archived sales.

```
select o.OrderID, o.Sales Current_Sales, oa.Sales Archive_Sales
from salesdb.orders as o
inner join salesdb.orders_archive as oa
on o.orderid = oa.orderid
where o.Sales != oa.Sales or oa.Sales is null;
```

Q36. Join Orders, Customers and Products. Find orders where Quantity is at least 2 and Sales is greater than 15. Display customer name, product name, quantity and sales.

```
select c.FirstName, p.Product, o.Quantity, o.Sales
from salesdb.orders as o
left join salesdb.customers as c
on o.customerid = c.customerid
left join salesdb.products as p
on o.productid = p.productid
where Quantity>=2 and Sales>15;
```

Q37. From Orders, find orders placed between 2025-01-01 and 2025-02-28, inclusive. Display the order ID, order date, status and sales. Sort by order date.

```
select o.OrderID, o.OrderDate, o.orderstatus , o.Sales
from salesdb.orders as o
where OrderDate >= '2025-01-01' and OrderDate < '2025-02-28'
order by OrderDate;
```

Q38. From Employees, calculate the number of employees and average salary for each department. Show only departments with an average salary greater than 60,000. Sort by average salary, highest first.

```
select e.Department, count(e.employeeid) Num_Employees,
round(avg(e.salary),2) Avg_Salary
from salesdb.employees as e
group by e.Department
having avg(e.salary) > 60000
order by Avg_Salary desc;
```

Q39. Use a LEFT JOIN between Products and Orders. For each category, display the number of products and total sales generated by orders. Include categories even if they have no orders.

```
select p.Category, count(distinct p.productid) Total_Products, sum(o.Sales)
Total_Sales
from salesdb.products as p
left join salesdb.orders as o
on p.productid = o.productid
group by Category;
```

Q40. Use Customers and Orders to find customers who have no delivered orders. Include customers who have no orders at all. Display their customer ID and name.

```
select o.Customerid, c.FirstName
from salesdb.customers as c
left join salesdb.orders as o
on c.customerid = o.customerid
where o.orderid is null;
```



Ankush Kumar

YOUR SQL JOURNEY DOESN'T END HERE.

This is just the beginning. Keep practising and keep building. 🚀

